

Mini-Projet B — Intégration numérique

Comparaison de méthodes et apport de NumPy

MGA802 — Été 2026

1 Problème et solution exacte

On cherche l'aire sous la courbe d'un polynôme de degré 3 sur un intervalle $[a, b]$:

$$I = \int_a^b f(x) dx, \quad f(x) = p_1 + p_2x + p_3x^2 + p_4x^3.$$

Pour les tests on prend $p = [2, -3, 1, 2]$ et $[a, b] = [-1, 2]$. Une primitive de f s'obtient terme à terme,

$$F(x) = p_1x + \frac{p_2}{2}x^2 + \frac{p_3}{3}x^3 + \frac{p_4}{4}x^4,$$

ce qui donne la valeur exacte $I_{\text{exact}} = F(b) - F(a) = 12$. Cette valeur sert de référence : l'erreur d'une méthode numérique est $|I_{\text{num}} - I_{\text{exact}}|$.

2 Les trois méthodes

On découpe $[a, b]$ en n segments de largeur $h = (b - a)/n$.

Rectangle (point milieu). On évalue f au centre de chaque segment : $I \approx h \sum_i f(x_i + \frac{h}{2})$.

Trapèze. On relie les valeurs aux bornes de chaque segment par une droite. La forme composite revient à donner un poids 1/2 aux deux extrémités et 1 aux nœuds intérieurs.

Simpson. On approxime f par une parabole passant par les deux bornes et le milieu de chaque segment :

$$\int_g^d f \approx \frac{h}{6} [f(g) + 4f(m) + f(d)].$$

Comme f est de degré 3, cette formule est *exacte* : Simpson retombe sur la valeur analytique, à l'arrondi machine près.

Chaque méthode est codée deux fois : une version Python de base (boucle `for` indexée) et une version NumPy où l'on construit tous les points en un seul tableau, puis on somme sans boucle Python.

3 Convergence : effet du nombre de segments

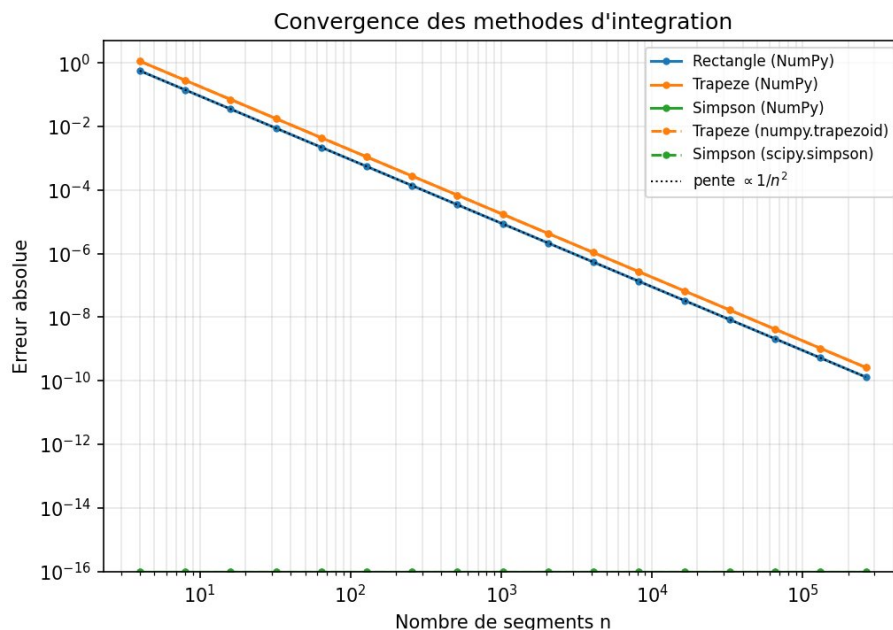


Figure 1: Erreur absolue en fonction de n (échelle log-log). Les courbes NumPy et pré-programmées se superposent à leurs versions maison.

La figure 1 montre deux comportements très différents. Le rectangle et le trapèze suivent une droite de pente -2 : leur erreur décroît comme $1/n^2$. Diviser h par deux divise donc l'erreur par environ quatre. Pour $n = 10$, on mesure une erreur de $9,0 \times 10^{-2}$ pour le rectangle et de $1,8 \times 10^{-1}$ pour le trapèze. Le rectangle (point milieu) est exactement deux fois plus précis que le trapèze à n égal, un résultat classique : les deux méthodes ont une erreur en h^2 , mais avec des constantes opposées et un facteur deux en faveur du point milieu.

Simpson, lui, reste plat tout en bas du graphique, autour de 10^{-15} , c'est-à-dire au niveau du bruit d'arrondi en double précision. Augmenter n ne l'améliore pas, parce qu'il n'y a déjà plus rien à gagner : la méthode est exacte pour un polynôme cubique. (Sur la figure, les valeurs nulles ou quasi nulles de Simpson sont ramenées à un plancher de 10^{-16} pour rester visibles en échelle logarithmique.)

4 Temps de calcul : Python de base contre NumPy

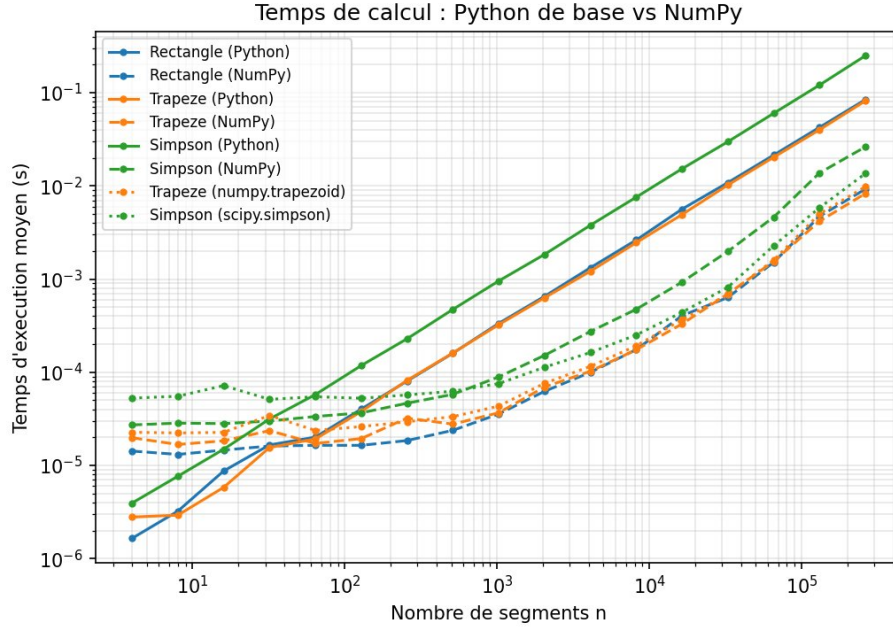


Figure 2: Temps d'exécution moyen (mesuré avec `timeit`) en fonction de n . Trait plein : Python de base ; tirets : NumPy ; pointillés : méthodes pré-programmées.

NumPy n'est pas systématiquement plus rapide (figure 2). À petit n , créer les tableaux coûte un temps fixe (de l'ordre de 10^{-5} s ici) qui domine, et la simple boucle Python s'en sort mieux. Le croisement se produit entre quelques dizaines et une centaine de segments selon la méthode : au-delà, les courbes NumPy passent nettement sous celles de Python et l'écart se creuse.

Table 1: Temps moyen par appel à $n = 262\,144$ segments (valeurs lues sur le graphe ; les chiffres exacts s'affichent dans le terminal au lancement).

Méthode	Python de base	NumPy	Accélération
Rectangle	$\approx 9 \times 10^{-2}$ s	$\approx 1 \times 10^{-2}$ s	$\approx \times 9$
Trapeze	$\approx 8 \times 10^{-2}$ s	$\approx 9 \times 10^{-3}$ s	$\approx \times 9$
Simpson	$\approx 2,3 \times 10^{-1}$ s	$\approx 2,5 \times 10^{-2}$ s	$\approx \times 9$

À grand n , la vectorisation fait gagner environ un facteur 9 (tableau 1). L'avantage de NumPy devient franc dès que n est grand : la boucle Python paie un coût à chaque itération que NumPy évite en travaillant directement sur le tableau entier, en code compilé. Simpson reste la méthode la plus lente des trois dans les deux versions, ce qui est logique : elle demande trois évaluations de f par segment contre une pour le rectangle ; mais elle profite du même gain en passant à NumPy. Les méthodes pré-programmées (pointillés) suivent de près les versions maison vectorisées. Les temps absolus dépendent de la machine ; ce sont les tendances et les rapports qui comptent.

5 Erreur par méthode et méthodes pré-programmées

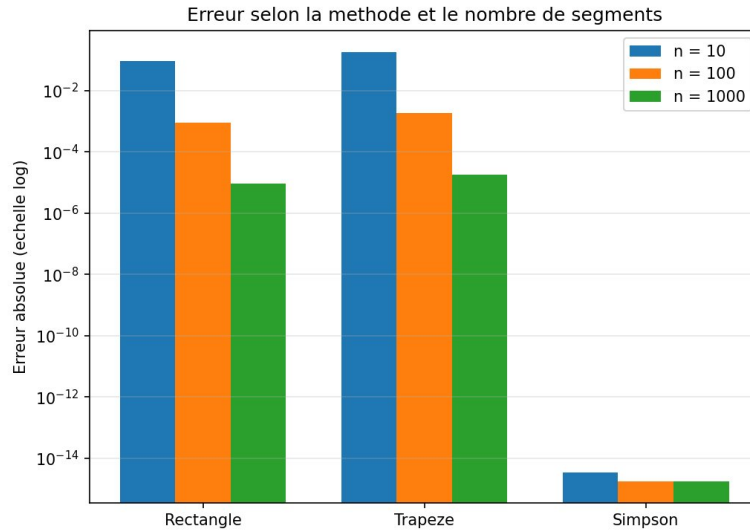


Figure 3: Erreur selon la méthode pour trois valeurs de n (échelle log).

La figure 3 résume le compromis. Pour le rectangle et le trapèze, chaque facteur 10 sur n fait gagner deux ordres de grandeur sur l'erreur, conformément à la loi en $1/n^2$. Simpson reste à 10^{-15} dans les trois cas : inutile d'augmenter n . On a vérifié nos implémentations avec deux fonctions toutes faites, `numpy.trapezoid` (ex-`numpy.trapz`) pour le trapèze et `scipy.integrate.simpson` pour Simpson. Elles renvoient exactement les mêmes valeurs que nos versions, d'où la superposition parfaite des courbes sur les figures 1 et 2.

6 Conclusion

Trois enseignements se dégagent. D'abord, pour les méthodes d'ordre fini (rectangle, trapèze), augmenter n améliore la précision mais coûte proportionnellement plus cher en temps : c'est un compromis direct. Ensuite, le choix de la méthode pèse beaucoup plus que n : Simpson atteint la précision machine dès $n = 1$ sur ce cas cubique, là où le trapèze a besoin de dizaines de milliers de segments pour s'en approcher. Enfin, la vectorisation NumPy apporte un vrai gain, mais seulement une fois passé un certain n ; en dessous, son coût fixe la pénalise. La bonne combinaison ici est sans hésiter Simpson en version NumPy : la plus précise, et rapide dès que n grandit.